

STA 445 Assignment 1

Problem 1

Create a vector of three elements (2,4,6) and name that vector `vec_a`. Create a second vector, `vec_b`, that contains (8,10,12). Add these two vectors together and name the result `vec_c`. Print only the result `vec_c`. Do not use the print command.

```
vec_a <- c(2,4,6)
vec_b <- c(8,10,12)
vec_c <- vec_a + vec_b
vec_c
```

```
## [1] 10 14 18
```

Problem 2

Create a vector, named `vec_d`, that contains only two elements (14,20). Add this vector to `vec_a` and show the result. What is the result and what do you think R did (look up the recycling rule using Google)? What is the warning message that R gives you?

```
vec_d <- c(14,20)
vec_d + vec_a
```

```
## Warning in vec_d + vec_a: longer object length is not a multiple of shorter
## object length
```

```
## [1] 16 24 20
```

It looks like R may have added 2+14, 4+20, then run out of `vec_d` and went back to the beginning, doing 14+6. The warning message is that longer object length is not a multiple of shorter object length, which is probably because the usual use case of this would be to do the same operation on a sequence, ie adding 2 to every the even numbers, adding 6 to the odd numbers, etc.

Problem 3

Next add 5 to the vector `vec_a` and show the result. What is the result and what did R do? Why doesn't R give you a warning message similar to what you saw in the previous problem?

```
vec_a + 5
```

```
## [1] 7 9 11
```

It adds 5 to each item. There is no warning message because 3 is a multiple of 1, so it gets through 3 full uses of the smaller object by the end.

Problem 4

Generate the vector of integers $\{1, 2, \dots, 5\}$ in two different ways.

- a. First using the `seq()` function

```
seq(1,5)
```

```
## [1] 1 2 3 4 5
```

- b. Using the `a:b` shortcut.

```
1:5
```

```
## [1] 1 2 3 4 5
```

Problem 5

Generate the vector of even numbers $\{2, 4, 6, \dots, 20\}$

- a. Using the `seq()` function

```
seq(2,20,2)
```

```
## [1] 2 4 6 8 10 12 14 16 18 20
```

- b. Using the `a:b` shortcut and some subsequent algebra.

```
1:10*2
```

```
## [1] 2 4 6 8 10 12 14 16 18 20
```

Problem 6

Generate a vector of 21 elements that are evenly spaced between 0 and 1 using the `seq()` command and name this vector `x`. Print the vector `x`.

```
x <- seq(0,1,length.out=21)
x
```

```
## [1] 0.00 0.05 0.10 0.15 0.20 0.25 0.30 0.35 0.40 0.45 0.50 0.55 0.60 0.65 0.70
## [16] 0.75 0.80 0.85 0.90 0.95 1.00
```

Problem 7

Generate the vector $\{2, 4, 8, 2, 4, 8, 2, 4, 8\}$ using the `rep()` command to replicate the vector `c(2,4,8)`.

```
rep(c(2,4,8),3)
```

```
## [1] 2 4 8 2 4 8 2 4 8
```

Problem 8

Generate the vector $\{2, 2, 2, 2, 4, 4, 4, 4, 8, 8, 8, 8\}$ using the `rep()` command. You might need to check the help file for `rep()` to see all of the options that `rep()` will accept. In particular, look at the optional argument `each=`.

```
rep(c(2,4,8),each=4)
```

```
## [1] 2 2 2 2 4 4 4 4 8 8 8 8
```

Problem 9

In this problem, we will work with the matrix

$$\begin{bmatrix} 2 & 4 & 6 & 8 & 10 \\ 12 & 14 & 16 & 18 & 20 \\ 22 & 24 & 26 & 28 & 30 \end{bmatrix}$$

- Follow the prompts below to create the matrix in two ways and save the resulting matrix as `M`.
- Create the matrix using some combination of the `seq()` and `matrix()` commands. Print `M`.

```
M <- matrix(data=seq(2,30,by=2),nrow=3,byrow=TRUE)
M
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    2    4    6    8   10
## [2,]   12   14   16   18   20
## [3,]   22   24   26   28   30
```

- ii. Create the same matrix by some combination of multiple `seq()` commands and either the `rbind()` or `cbind()` command. Print M

```
M <- rbind(1:5*2, 6:10*2, 11:15*2)
M
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    2    4    6    8   10
## [2,]   12   14   16   18   20
## [3,]   22   24   26   28   30
```

- b. Extract the second row out of M.

```
M[2,]
```

```
## [1] 12 14 16 18 20
```

- c. Extract the element in the third row and second column of M.

```
M[3,2]
```

```
## [1] 24
```

Problem 10

The following code creates a `data.frame` and then has two different methods for removing the rows with NA values in the column `Grade`. Explain the difference between the two.

```
df <- data.frame(name= c('Alice', 'Bob', 'Charlie', 'Daniel'),
                  Grade = c(6,8,NA,9))

df[ -which( is.na(df$Grade) ), ]

df[ which( !is.na(df$Grade) ), ]
```

These methods have identical results. The first asks for the inverse of all rows where Grade is NA, while the second asks for all rows where Grade is not NA.

Problem 11

Create and manipulate a list.

- a. Create a list named `my.test` with elements $x = c(4,5,6,7,8,9,10)$ $y = c(34,35,41,40,45,47,51)$ $slope = 2.82$ $p.value = 0.000131$

To show you have created your list correctly, type use `str()` to show the structure of the list.

```
my.test <- list(x=4:10,y=c(34,35,41,40,45,47,51),slope=2.82,p.value=0.000131)
str(my.test)
```

```
## List of 4
## $ x      : int [1:7] 4 5 6 7 8 9 10
## $ y      : num [1:7] 34 35 41 40 45 47 51
## $ slope  : num 2.82
## $ p.value: num 0.000131
```

b. Extract the second element in the list.

```
my.test[2]
```

```
## $y
## [1] 34 35 41 40 45 47 51
```

c. Extract the element named `p.value` from the list.

```
my.test['p.value']
```

```
## $p.value
## [1] 0.000131
```